

Estructura del Proyecto en Prolog — Sistema Experto de Recomendación de Mascotas

Objetivo

Definir una estructura modular para el sistema experto de recomendación de mascotas utilizando Prolog. La finalidad es separar correctamente:

- hechos,
- preguntas,
- reglas,
- inferencia,
- y flujo principal.

Esto facilitará:

- mantenimiento,
- escalabilidad,
- depuración,
- y comprensión del sistema.

Estructura General del Proyecto

```
proyecto_mascotas/  
|  
├─ main.pl  
├─ mascotas.pl  
├─ preguntas.pl  
├─ respuestas.pl  
├─ reglas.pl  
├─ inferencia.pl  
└─ utilidades.pl
```

1. main.pl

Propósito

Archivo principal del sistema.

Se encarga de:

- iniciar el programa,

- cargar archivos,
 - ejecutar preguntas,
 - llamar al motor de inferencia,
 - mostrar resultados.
-

Estructura sugerida

```
:- consult(mascotas).  
:- consult(preguntas).  
:- consult(respuestas).  
:- consult(reglas).  
:- consult(inferencia).  
:- consult(utilidades).
```

```
inicio :-  
    limpiar_respuestas,  
    hacer_preguntas,  
    recomendar.
```

Responsabilidades

Cargar módulos

```
:- consult(nombre_archivo).
```

Iniciar flujo

```
inicio.
```

Reiniciar respuestas anteriores

```
limpiar_respuestas.
```

Ejecutar preguntas

```
hacer_preguntas.
```

Obtener recomendación final

```
recomendar .
```

2. mascotas.pl

Propósito

Definir todas las mascotas disponibles en el sistema.

Estructura sugerida

```
mascota(perro).  
mascota(gato).  
mascota(conejo).  
mascota(hamster).  
mascota(pez).  
mascota(tortuga).  
mascota(ave).
```

Función

Este archivo representa el conjunto inicial de opciones.

Todas las mascotas empiezan como posibles hasta que las reglas las descarten.

3. preguntas.pl

Propósito

Contener todas las preguntas que el sistema realizará al usuario.

Este archivo controla:

- preguntas,
 - lectura de respuestas,
 - almacenamiento de respuestas.
-

Estructura sugerida

```
hacer_preguntas :-  
    preguntar_alergias,  
    preguntar_espacio,  
    preguntar_ninos,  
    preguntar_otras_mascotas,  
    preguntar_tiempo,  
    preguntar_presupuesto,  
    preguntar_experiencia,  
    preguntar_limpieza,  
    preguntar_responsabilidad,  
    preguntar_actividad,  
    preguntar_convivencia,  
    preguntar_ruido.
```

Ejemplo de pregunta

```
preguntar_alergias :-  
    write('¿Hay alergias al pelo o plumas? (si/no)'), nl,  
    read(Respuesta),  
    assert(respuesta(alergias, Respuesta)).
```

Otra pregunta

```
preguntar_espacio :-  
    write('¿Qué nivel de espacio tienes?'), nl,  
    write('(bajo/medio/alto)'), nl,  
    read(Respuesta),  
    assert(respuesta(espacio, Respuesta)).
```

Función

Guardar respuestas dinámicamente.

Ejemplo:

```
respuesta(alergias, si).  
respuesta(espacio, pequeno).
```

4. respuestas.pl

Propósito

Manejar almacenamiento y limpieza de respuestas.

Estructura sugerida

```
:- dynamic respuesta/2.
```

Limpiar respuestas

```
limpiar_respuestas :-  
    retractall(respuesta(_, _)).
```

Función

Permitir que el sistema:

- guarde respuestas del usuario,
 - modifique respuestas,
 - reinicie el sistema.
-

5. reglas.pl

Propósito

Contener TODAS las reglas de descarte.

Este archivo representa la base de conocimiento del sistema experto.

Filosofía de funcionamiento

Cada regla elimina mascotas incompatibles.

NO se selecciona directamente una mascota.

El sistema funciona descartando opciones.

Estructura base

```
descartar(Mascota) :-  
    condicion.
```

Reglas por categoría

Alergias

```
descartar(perro) :-  
    respuesta(alergias, si).  
  
descartar(gato) :-  
    respuesta(alergias, si).  
  
descartar(conejo) :-  
    respuesta(alergias, si).  
  
descartar(ave) :-  
    respuesta(alergias, si).
```

Espacio pequeño

```
descartar(perro) :-  
    respuesta(espacio, bajo).
```

Poco tiempo

```
descartar(perro) :-  
    respuesta(tiempo, poco).  
  
descartar(conejo) :-  
    respuesta(tiempo, poco).
```

Poco presupuesto

```
descartar(perro) :-  
    respuesta(presupuesto, bajo).
```

Baja tolerancia a limpieza

```
descartar(conejo) :-  
    respuesta(limpieza, si).
```

No tolera ruido

```
descartar(ave) :-  
    respuesta(ruido, si).
```

6. inferencia.pl

Propósito

Contener el motor de inferencia.

Determina:

- mascotas válidas,
 - mascotas descartadas,
 - resultado final.
-

Obtener mascotas recomendadas

```
recomendada(Mascota) :-  
    mascota(Mascota),  
    not(descartar(Mascota)).
```

Mostrar recomendaciones

```
recomendar :-  
    findall(M, recomendada(M), Lista),  
    mostrar_resultado(Lista).
```

Resultado sin mascotas

```
mostrar_resultado([]) :-  
    write('No se recomienda una mascota con las condiciones actuales.').
```

Resultado con una mascota

```
mostrar_resultado([Mascota]) :-  
    write('Mascota recomendada: '),  
    write(Mascota).
```

Resultado con varias mascotas

```
mostrar_resultado(Lista) :-  
    write('Mascotas compatibles: '), nl,  
    write(Lista).
```

7. utilidades.pl

Propósito

Funciones auxiliares.

Posibles utilidades

Validación de respuestas

```
validar_si_no(si).  
validar_si_no(no).
```

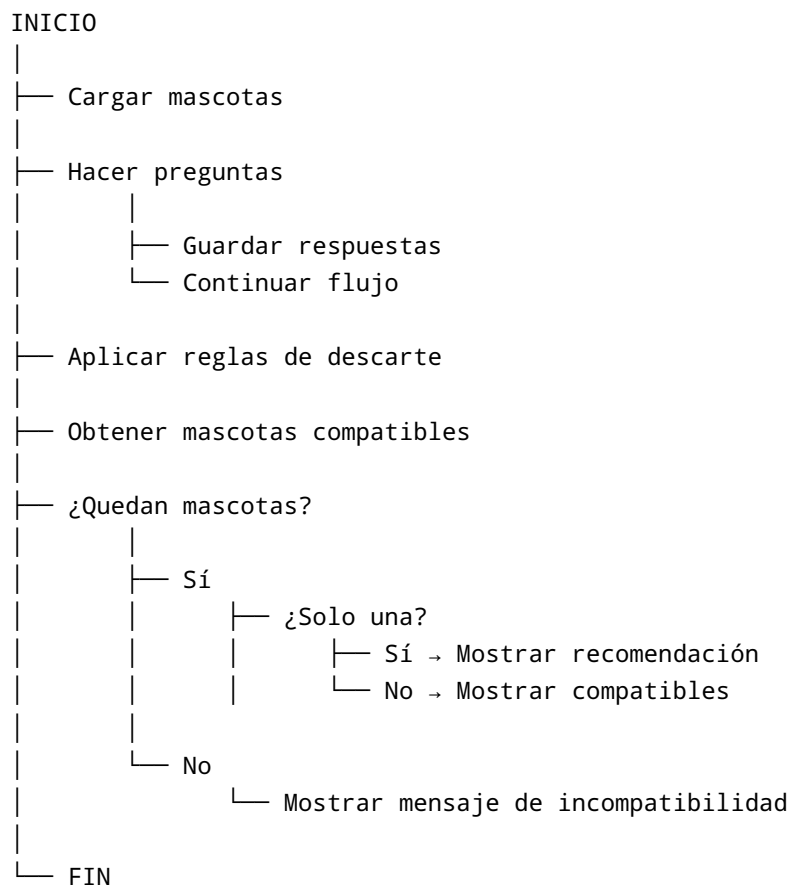
Mostrar separadores

```
separador :-  
    write('-----'), nl.
```

Limpiar pantalla

Opcional dependiendo del entorno Prolog.

Flujo Completo del Sistema



Cómo Funciona el Descarte

Estado inicial

[perro, gato, conejo, hamster, pez, tortuga, ave]

Ejemplo 1

Usuario:

- alergias = sí

Reglas aplicadas:

Eliminar:

- perro
- gato
- conejo
- ave

Resultado:

[hamster, pez, tortuga]

Ejemplo 2

Usuario:

- tiempo = poco

Reglas aplicadas:

Eliminar:

- hamster

Resultado:

[pez, tortuga]

Ejemplo 3

Usuario:

- convivencia = baja

Resultado final:

[pez]

Ventajas de esta Arquitectura

Modularidad

Cada archivo tiene una responsabilidad específica.

Escalabilidad

Es fácil agregar:

- nuevas mascotas,
 - nuevas preguntas,
 - nuevas reglas.
-

Mantenimiento

Las reglas están separadas del flujo principal.

Claridad lógica

El sistema funciona mediante inferencia y descarte.

Compatibilidad con Prolog

La estructura aprovecha:

- hechos,
- reglas,
- consultas,
- inferencia lógica.

Posibles Mejoras Futuras

Sistema de puntuación

En lugar de descartar completamente.

Interfaz gráfica

Conectar Prolog con:

- Python,
 - Java,
 - HTML,
 - o Electron.
-

Explicación de resultados

Ejemplo:

"Se recomienda gato porque:

- tienes espacio medio,
 - tiempo moderado,
 - buscas convivencia alta."
-

Reglas avanzadas

Permitir:

- múltiples niveles de compatibilidad,
 - recomendaciones secundarias,
 - restricciones más complejas.
-

Conclusión

La arquitectura propuesta organiza correctamente el sistema experto en módulos independientes, permitiendo implementar inferencia lógica mediante descarte progresivo de mascotas incompatibles.

La estructura facilita la implementación en Prolog, reduce complejidad y permite expandir el sistema de manera ordenada.